HeatTransfer3D: A High-Performance Multi-Backend Solver for 3D Heat Conduction with STL Geometry Support

20 April 2026

Summary

`HeatTransfer3D` is a lightweight, high-performance C++ solver for 3D heat conduction problems on structured Cartesian grids. It imports 3D geometries from STL files and incorporates them into the computational domain using voxelization. Geometry is mapped onto grid cells and used to define internal, external, and boundary regions. The solver supports mixed Dirichlet and Neumann boundary conditions on these regions.

The software provides **four interchangeable solver backends** — CPU and CUDA implementations of stencil-based and matrix-based Jacobi iterative solvers — allowing users to balance computational speed, memory usage, and hardware availability. Results are exported in VTK format for visualization in ParaView.

The software targets researchers and engineers working in thermal management, materials processing, electronics cooling, and related heat conduction applications.

Statement of Need

Modeling three-dimensional heat conduction in complex geometries is a common requirement in applications such as electronics cooling, battery thermal management, additive manufacturing, and heat exchanger design. Despite its importance, existing tools often involve trade-offs between ease of use, computational efficiency, and geometric modeling complexity.

Commercial multiphysics platforms such as ANSYS and COMSOL Multiphysics provide comprehensive capabilities but are often inaccessible due to licensing costs and workflow complexity. Open-source frameworks such as OpenFOAM (Weller et al. 2007; Jasak, Jemcov, and Tukovic 2007) and FEniCS (Logg, Mardal, and Wells 2012) offer high flexibility and general-purpose multiphysics functionality,

but typically require substantial setup effort, including mesh generation and solver configuration, even for pure heat conduction problems.

In contrast, many lightweight finite-difference solvers are computationally efficient but are generally restricted to structured Cartesian grids and assume that the computational geometry is already represented on a grid. While GPU-accelerated implementations exist, they are typically focused on stencil optimization and performance improvement rather than providing an integrated workflow that connects complex geometries to structured-grid heat solvers.

`HeatTransfer3D` addresses this gap by providing a lightweight, focused, and high-performance open-source solver for steady-state and transient 3D heat conduction. The key feature of the software is the integration of geometry into a structured-grid framework using STL-based voxelization, where complex surfaces are mapped onto a Cartesian grid using ray tracing.

Key features include: - Geometry handling via STL import with voxelization-based domain construction, - Support for mixed Dirichlet and Neumann boundary conditions, - Four interchangeable solver backends (CPU and CUDA implementations of stencil-based and matrix-based Jacobi solvers), enabling flexible trade-offs between performance, memory usage, and hardware availability, - Simple command-line interface and VTK output for visualization in ParaView.

This makes the software suitable for rapid prototyping, parametric studies, and engineering applications where reproducibility, computational efficiency, and minimal setup overhead are critical.

State of the Field

Existing tools for heat conduction simulation can be broadly categorized into three groups: general-purpose multiphysics frameworks, mesh-based finite element tools, and specialized research codes.

General-purpose frameworks such as OpenFOAM (Weller et al. 2007; Jasak, Jemcov, and Tukovic 2007) and FEniCS (Logg, Mardal, and Wells 2012) provide extensive flexibility and support for coupled physics problems. However, they typically require substantial configuration effort, including mesh generation, solver selection, and case setup, which can be excessive for problems focused solely on heat conduction.

On the other end of the spectrum, lightweight finite-difference and finite-volume solvers are computationally efficient and widely used for heat conduction and diffusion-dominated problems due to their straightforward implementation on structured Cartesian grids (LeVeque 2007; Patankar 1980). More advanced structured-grid frameworks, such as OpenSBLI (Howell and others 2016), extend this class of methods to high-performance computing environments. However, such frameworks are designed as general-purpose PDE toolchains and typically

require additional abstraction layers and configuration to define application-specific workflows.

Several GPU-accelerated finite-difference approaches have been developed to improve performance on structured grids. For example, phase-change heat conduction simulations on GPUs demonstrate substantial acceleration through optimized stencil execution, while remaining based on structured-grid discretizations that require complex geometries to be represented in grid-aligned form rather than directly incorporated from surface-based representations (Liu et al. 2022). Similarly, fast and interactive GPU-based heat conduction simulators have been proposed for two-dimensional problems, prioritizing real-time performance and interactivity over geometric modeling flexibility (Wei and others 2014). These approaches are typically designed around specific research applications and optimized stencil implementations, and do not provide modular solver architectures for extensible simulation workflows.

In addition, simplified and educational solvers such as FDiff3 emphasize numerical understanding and clarity of implementation for heat conduction problems, primarily in pedagogical settings (Russell and Probert 2004). Similarly, Python-based educational frameworks for two-dimensional heat transfer focus on accessibility and teaching purposes rather than extensible or modular solver design.

Meshless approaches such as radial basis function finite differences (RBF-FD) enable simulation on scattered nodes and can represent arbitrary three-dimensional geometries without structured grids (Fornberg and Flyer 2015; Miotti and others 2021). However, these methods are typically CPU-based, involve increased formulation complexity, and require careful parameter selection.

HeatTransfer3D occupies a middle ground between these categories by combining:

- geometry-driven preprocessing that maps STL-based surfaces onto a structured Cartesian grid through voxelization,
- the simplicity and efficiency of finite-difference discretization on regular grids,
- a multi-backend design enabling both CPU and GPU execution,
- a lightweight implementation focused specifically on heat conduction problems.

This combination positions it as a focused engineering tool that integrates geometry-to-grid workflow handling with structured-grid solvers, rather than as a general-purpose multiphysics framework or an educational prototype.

Software Design

The codebase follows a modular structure written in C++17 with optional CUDA support. The main directories are organized as follows:

- **`src/`**: Core implementation files, including the main driver, geometry processing, boundary condition handling, solver kernels, and VTK output routines.
- **`include/`**: Header files defining classes and functions for the grid, solvers, STL importer, and utilities.
- **`tests/`**: Unit and integration tests.
- **`stlFiles/`**: Sample geometries (cube, cylinder, L_Channel, semiCylinder) with associated boundary patch files.

The solver uses a uniform Cartesian grid with finite-difference discretization of the heat equation. Two fundamental algorithmic approaches are implemented:

1. **Stencil-based solvers** — Direct Jacobi iterations on the temperature field (low memory footprint, high performance).
2. **Matrix-based solvers** — Explicit assembly of a sparse coefficient matrix followed by Jacobi iterations (more flexible for extensions).

Each approach has both a CPU and a CUDA (GPU) implementation. Users can select any of the four backends at runtime using a simple command-line flag (`--solver 1..4`). CUDA kernels are optimized for coalesced memory access and make use of shared memory where beneficial. The geometry pipeline automatically detects and labels boundary patches from separate STL files, simplifying the application of different boundary conditions.

CMake is used for building, with convenient `build.sh` and `installDeps` scripts provided.

Performance

One of the key strengths of `HeatTransfer3D` is its **multi-backend design**, which allows users to dynamically select the most suitable solver based on the available hardware and problem size.

Performance comparisons for a representative test case (cube geometry with steady-state convergence) are shown below:

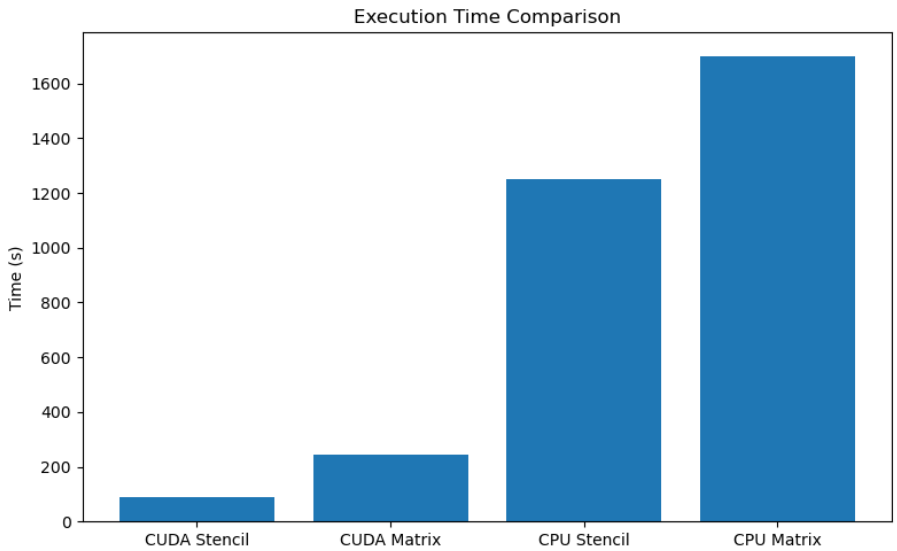

Figure 1: Execution time comparison of the four solvers for a $100 \times 100 \times 100$ grid (lower is better). CUDA backends demonstrate substantial speedups over CPU implementations.

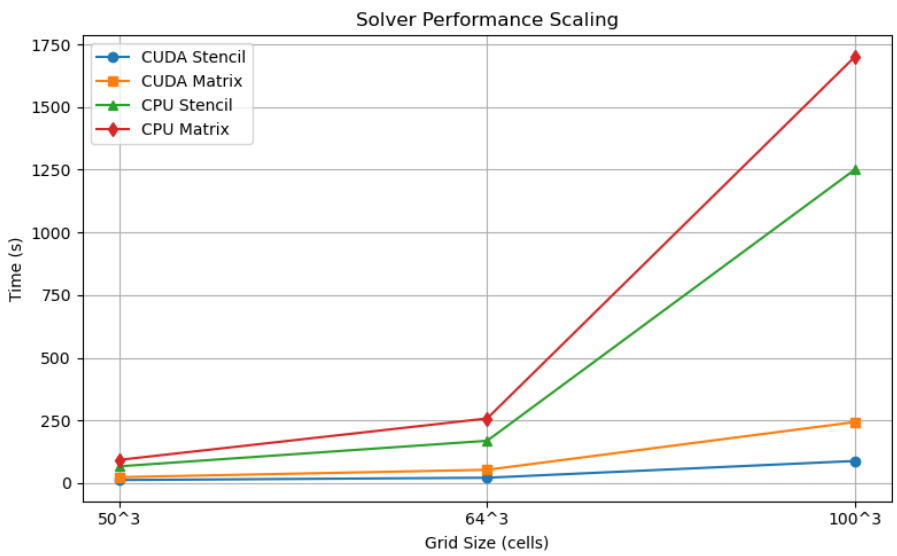

Figure 2: Strong scaling with increasing grid resolution (50^3 to 150^3). The GPU stencil backend maintains strong performance due to its low memory overhead.

All tests were conducted on the following hardware: CPU — 11th Gen Intel® Core™ i5-11400H @ 2.70 GHz; GPU — NVIDIA GeForce RTX 3050. The global tolerance was set as 1×10^{-6}. The CUDA stencil solver achieves an **8–20**×

speedup compared to the CPU stencil solver, while still providing reliable performance on CPU backends. This flexibility makes the software suitable for both rapid prototyping and large-scale simulations.

Simulations

Here, the few shapes simulated by the solver will be displayed

Research Impact Statement

By offering high performance together with flexible solver selection, `HeatTransfer3D` enables faster design iterations and parametric studies in thermal engineering research. The open-source MIT license and modular structure also facilitate community contributions and integration into larger multiphysics workflows.

AI Usage Disclosure

AI-based tools were used to assist with debugging, syntax support, and language refinement during the development and writing of this work. All scientific decisions, implementation design, and results validation were performed and verified by the authors.

The following tools were used: - ChatGPT (GPT-4o) - Grok

Acknowledgements

The author thanks the open-source community for the tools that made this project possible.

References

Fornberg, Bengt, and Natasha Flyer. 2015. *Solving Pdes with Radial Basis Functions*. Cambridge University Press.

Howell, Jonathan, and others. 2016. "OpenSBLI: An Open-Source Framework for the Automated Derivation and Parallel Execution of Finite Difference Solvers on Structured Grids." *Computer Physics Communications*.

Jasak, Hrvoje, Aleksandar Jemcov, and Zeljko Tukovic. 2007. "OpenFOAM: A C++ Library for Complex Physics Simulations." *International Workshop on Coupled Methods in Numerical Dynamics*, 1–20.

LeVeque, Randall J. 2007. *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM.

Liu, Xiao-Yu, Zhi Xie, Jian Yang, and Hong-Ji Meng. 2022. "Accelerating Phase-Change Heat Conduction Simulations on Gpus." *Case Studies in Thermal Engineering* 39: 102410.

Logg, Anders, Kent-Andre Mardal, and Garth N. Wells. 2012. *Automated Solution of Differential Equations by the Finite Element Method*. Springer.

Miotti, D., and others. 2021. "A Fully Meshless Approach to the Numerical Simulation of Heat Conduction Problems over Arbitrary 3D Geometries." *Energies* 14 (5): 1351. https://doi.org/10.3390/en14051351.

Patankar, Suhas V. 1980. *Numerical Heat Transfer and Fluid Flow*. Hemisphere Publishing Corporation.

Russell, MB, and SD Probert. 2004. "FDiff3: A Finite-Difference Solver for Facilitating Understanding of Heat Conduction and Numerical Analysis." *Applied Energy* 79 (4): 443–56.

Wei, Z., and others. 2014. "A Fast and Interactive Heat Conduction Simulator on Gpus." *Journal of Computational and Applied Mathematics*.

Weller, Henry G., G. Tabor, Hrvoje Jasak, and C. Fureby. 2007. "A Tensorial Approach to Computational Continuum Mechanics Using Object-Oriented Techniques." *Computers in Physics* 12 (6): 620–31.